

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Курсовая работа

по дисциплине: «Разработка мобильных приложений»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

Доцент, к.т.н. _____ Домашнев П.А.

Липецк, 2026

1. Аннотация

С. 27, Ил. 2, Литература 5 назв.

В данной курсовой работе приводится описание используемых средств, проектируется клиентское мобильное приложение ведения заметок "Todo"для iOS, реализуется его графический интерфейс, а также серверное приложение, реализующее REST API.

Содержание

1. Аннотация 2

2. Задание кафедры 5

3. Разработка серверного приложения 6

3.1 Назначение серверного приложения 6

3.2 Используемые технологии 6

3.3 Архитектура серверного приложения 7

3.4 Модель данных 8

3.5 Организация хранения данных 8

3.6 Реализация REST API 9

3.7 Асинхронная обработка запросов 10

3.8 Конфигурация сервера 11

4. Разработка графического интерфейса клиентского приложения 12

4.1 Назначение графического интерфейса 12

4.2 Архитектура графического интерфейса 12

4.3 Экран списка задач 13

4.4 Экран категорий 16

4.5 Использование Auto Layout 16

4.6 Организация пользовательских компонентов 17

4.7 Синхронизация интерфейса с данными 18

4.8 Особенности пользовательского интерфейса 18

5. Организация работы клиента с локальным и серверным хранилищем данных 19

5.1 Общая схема хранения данных 19

5.2 Работа с локальной базой данных 19

5.3 Структура локальной базы данных 20
5.4 Операции над локальными данными 21
5.5 Взаимодействие с серверным хранилищем 21
5.6 Получение данных с сервера 22
5.7 Добавление и изменение задач на сервере 23
5.8 Удаление данных с сервера 23
5.9 Механизм синхронизации данных 24
6. Список источников 25
7. Вывод 26

2. Задание кафедры

Курсовой проект состоит в написании мобильного приложения на платформе iOS. Тема для мобильного приложения выбирается студентом самостоятельно. Мобильное приложение должно удовлетворять следующим требованиям: 1) Приложение должно иметь собственный сервер, который используется для централизованного хранения данных (интерфейс сервера REST API) 2) На устройстве должна использоваться локальная база (CoreData, SQLite) 3) Визуальная часть приложения должна иметь минимум 5 экранов, в том числе обязательно использование TableViewController и CollectionViewController 4) Верстка должна быть выполнена с применением Autolayout и Constraints 5) Использование UserDefaults/Keychain для хранения пользовательских данных 6) Опционально работа с оборудованием (датчиками или камерой)

3. Разработка серверного приложения

3.1. Назначение серверного приложения

Серверное приложение предназначено для централизованного хранения пользовательских заметок мобильного приложения “Todo”, а также обеспечения доступа к ним посредством REST API. Наличие серверной части позволяет реализовать единое хранилище данных, доступное для клиентского приложения, что обеспечивает синхронизацию информации и сохранность пользовательских данных.

Основной задачей сервера является обработка HTTP-запросов, предоставление интерфейса для создания, получения, обновления и удаления задач, а также поддержка обмена данными в формате JSON.

Сервер реализован на языке программирования Swift с использованием серверного фреймворка Vapor[1]. Данный фреймворк предоставляет средства для создания REST API, обработки HTTP-запросов, сериализации данных и организации асинхронной логики приложения.

3.2. Используемые технологии

При разработке серверного приложения использовались следующие технологии:

- язык программирования Swift;
- серверный фреймворк Vapor;
- протокол HTTP для взаимодействия между клиентом и сервером;
- архитектурный стиль REST;
- формат сериализации данных JSON;

- механизм асинхронного выполнения Swift Concurrency.

Фреймворк Vapor предоставляет высокоуровневые механизмы маршрутизации HTTP-запросов, кодирования и декодирования JSON-структур, а также поддержку асинхронной обработки запросов, что позволяет создавать производительные серверные приложения.

3.3. Архитектура серверного приложения

Серверное приложение построено по модели REST API. Взаимодействие между клиентской и серверной частью осуществляется посредством HTTP-запросов.

В сервере реализованы следующие операции:

- получение списка задач;
- получение отдельной задачи по идентификатору;
- создание новой задачи;
- обновление существующей задачи;
- удаление задачи.

Для представления данных используется структура `TaskDTO`, которая выполняет роль объекта передачи данных (Data Transfer Object). Структура содержит информацию о задаче и используется для сериализации в JSON-формат.

Состав модели задачи включает:

- уникальный идентификатор задачи;
- заголовок;

- текстовое описание;
- признак выполнения;
- категорию;
- дату последнего изменения.

3.4. Модель данных

Для хранения информации о задаче используется структура `TaskDTO`, реализующая протокол `Content`. Поддержка данного протокола обеспечивает автоматическое преобразование объектов в JSON и обратно.

Структура задачи представлена следующими полями:

- `id` — уникальный идентификатор типа UUID;
- `title` — название задачи;
- `notes` — текстовое описание;
- `isDone` — логическое значение, отражающее статус выполнения;
- `category` — категория задачи;
- `updatedAt` — дата последнего изменения.

Использование идентификатора UUID позволяет гарантировать уникальность каждой задачи независимо от клиента, создавшего объект.

3.5. Организация хранения данных

Для хранения задач используется специальный компонент `TaskStorage`, реализованный в виде `actor`. Данный подход позволяет обеспечить потоко-

безопасный доступ к данным при одновременном выполнении нескольких запросов.

Хранилище поддерживает следующие операции:

- получение полного списка задач;
- поиск задачи по идентификатору;
- добавление или обновление записи;
- удаление задачи.

Данные сохраняются в файл формата JSON (`tasks.json`), расположенный в рабочем каталоге серверного приложения. При запуске сервера выполняется загрузка ранее сохранённых задач, после чего они помещаются во внутреннюю коллекцию.

После изменения состояния хранилища выполняется сериализация данных и их запись в файл. Для предотвращения повреждения данных применяется атомарная запись.

Использование файлового хранения вместо полноценной системы управления базами данных обусловлено сравнительно небольшим объёмом данных и учебным характером проекта.

3.6. Реализация REST API

Для взаимодействия с мобильным клиентом реализован REST API, поддерживающий основные HTTP-методы.

Маршруты сервера представлены в таблице 1.

Таблица 1: Маршруты REST API серверного приложения

Маршрут	Метод	Назначение
`/health`	GET	Проверка доступности сервера
`/tasks`	GET	Получение списка всех задач
`/tasks/{id}`	GET	Получение задачи по идентификатору
`/tasks`	POST	Создание новой задачи
`/tasks/{id}`	PUT	Обновление существующей задачи
`/tasks/{id}`	DELETE	Удаление задачи

Взаимодействие осуществляется посредством JSON-документов. При создании или обновлении задачи клиент передаёт структуру данных, которая автоматически декодируется сервером.

В случае ошибок используются HTTP-коды состояния. Например:

- `200 OK` — успешное выполнение запроса;
- `201 Created` — успешное создание объекта;
- `204 No Content` — успешное удаление объекта;
- `400 Bad Request` — ошибка параметров запроса;
- `404 Not Found` — объект не найден.

3.7. Асинхронная обработка запросов

Серверное приложение использует механизм асинхронного выполнения Swift Concurrency. Для предотвращения конфликтов доступа к данным компонент хранения реализован как `actor`, что позволяет гаран-

тировать корректную работу при конкурентном выполнении нескольких HTTP-запросов.

Применение асинхронной модели обработки позволяет серверу одновременно обслуживать несколько клиентов без блокировки потока выполнения.

3.8. Конфигурация сервера

Сервер запускается в режиме разработки (`development`) и прослушивает сетевой интерфейс по заданному IP-адресу и порту.

Для доступа используется порт `8080`. Такое решение позволяет мобильному приложению подключаться к серверу в локальной сети при тестировании функциональности.

На рисунке 1 представлен сервер, запущенный в эмуляторе терминала.

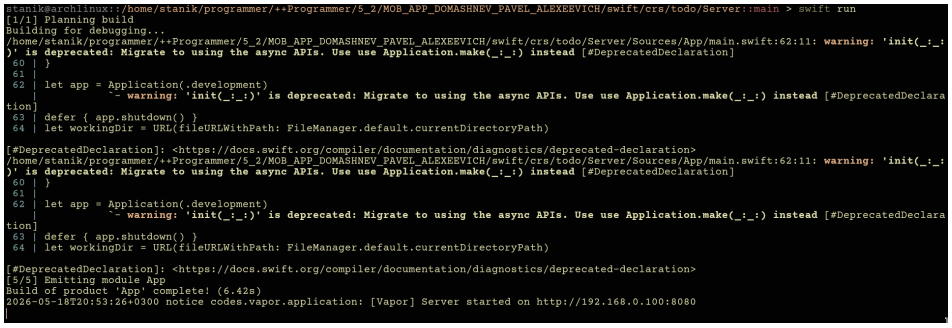


Рисунок 1 — Сервер

4. Разработка графического интерфейса клиентского приложения

4.1. Назначение графического интерфейса

Графический интерфейс мобильного приложения предназначен для взаимодействия пользователя с системой управления заметками “Todo”. Интерфейс обеспечивает создание, просмотр, редактирование, удаление и категоризацию задач, а также предоставляет средства навигации между различными разделами приложения.

Клиентская часть разработана для платформы iOS с использованием фреймворка UIKit. Данный подход позволяет реализовать нативный пользовательский интерфейс, соответствующий требованиям платформы Apple и обеспечивающий высокую производительность приложения.

Основу пользовательского интерфейса составляют экран отображения списка задач, экран категорий, экран просмотра подробной информации о задаче и экран добавления новой задачи.

4.2. Архитектура графического интерфейса

Построение пользовательского интерфейса выполнено на основе паттерна навигации `UINavigationController`, который обеспечивает переход между экранами приложения.

Каждый экран реализован отдельным контроллером представления. Такой подход позволяет изолировать логику отображения и упростить поддержку приложения.

Навигация между экранами осуществляется посредством последовательного перехода между контроллерами, что соответствует типичной архитектуре iOS-приложений.

4.3. Экран списка задач

Основным экраном приложения является экран списка задач, реализованный посредством класса `TasksTableViewController`, наследуемого от `UITableViewController`.

Данный экран предназначен для отображения списка пользовательских задач и реализует следующие функции:

- отображение списка всех задач;
- переход к экрану подробной информации;
- добавление новой задачи;
- удаление существующей задачи;
- синхронизация данных с сервером.

Для отображения данных используется компонент `UITableView`. Каждая строка таблицы содержит:

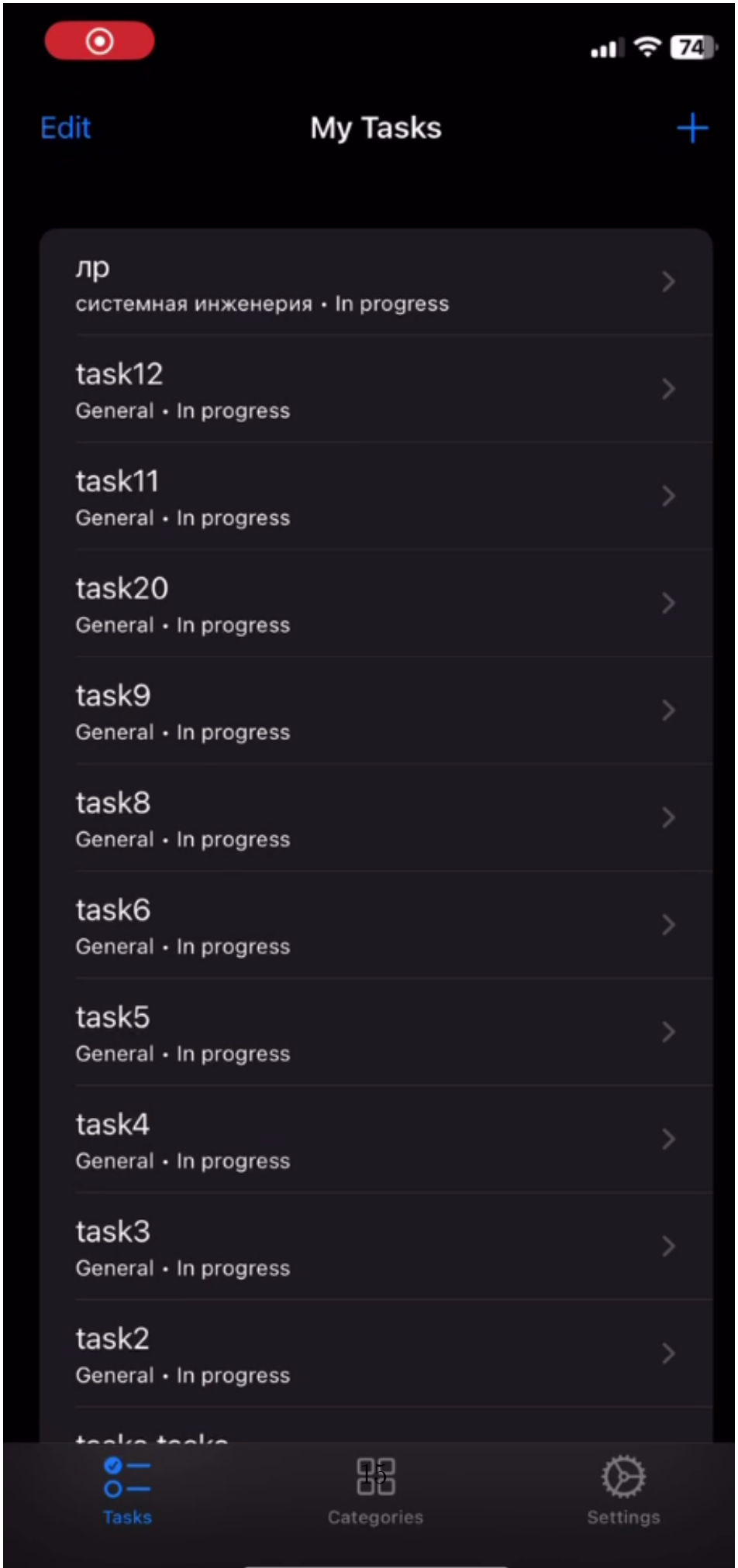
- название задачи;
- категорию;
- состояние выполнения.

При выборе строки выполняется переход на экран детальной информации о задаче.

Для добавления новой записи используется системная кнопка `UIBarButtonItem` с типом `add`, расположенная в панели навигации. При её выборе открывается экран создания задачи.

Также реализован механизм удаления элементов посредством встроенного режима редактирования таблицы. Удаление задачи приводит к её исключению из локального хранилища и отправке запроса удаления на сервер.

На рисунке 2 представлен экран списка задач клиентского приложения.


Edit
My Tasks
лр
системная инженерия • In progress
task12
General • In progress
task11
General • In progress
task20
General • In progress
task9
General • In progress
task8
General • In progress
task6
General • In progress
task5
General • In progress
task4
General • In progress
task3
General • In progress
task2
General • In progress
Tasks
Categories
Settings

4.4. Экран категорий

Для представления информации о распределении задач по категориям реализован экран категорий, представленный классом `CategoriesCollectionVi`

Данный экран построен на основе компонента `UICollectionViewController`, что удовлетворяет требованиям задания кафедры по использованию `CollectionViewController`.

Экран предназначен для отображения статистической информации о категориях задач. Для каждой категории отображаются:

- название категории;
- количество задач, относящихся к данной категории.

Список категорий формируется автоматически на основе имеющихся пользовательских задач посредством группировки данных.

Для визуального представления используется пользовательская ячейка `CategoryCell`, содержащая текстовые элементы интерфейса:

- название категории;
- количество задач.

Каждая ячейка имеет скруглённые границы и использует системные стили отображения iOS, что обеспечивает визуальную согласованность интерфейса.

4.5. Использование Auto Layout

Построение интерфейса выполнено с использованием технологии Auto Layout и системы ограничений (`Constraints`).

Для размещения элементов внутри пользовательских ячеек применяются ограничения, задаваемые программным способом посредством `NSLayoutConstraint`.

Например, элементы интерфейса внутри ячейки категории располагаются при помощи вертикального контейнера `UIStackView`, а позиционирование осуществляется через ограничения:

- выравнивание по левому краю;
- выравнивание по правому краю;
- центрирование по вертикали.

Применение Auto Layout позволяет обеспечить корректное масштабирование интерфейса на устройствах с различными размерами экрана и ориентацией.

4.6. Организация пользовательских компонентов

Для отображения информации используются стандартные компоненты UIKit:

- `UITableView` — отображение списка задач;
- `UICollectionView` — отображение категорий;
- `UILabel` — вывод текстовой информации;
- `UIStackView` — группировка элементов интерфейса;
- `UIBarButtonItem` — реализация навигационных действий.

Использование стандартных компонентов платформы обеспечивает единообразие визуального оформления и соответствие рекомендациям Human Interface Guidelines.

4.7. Синхронизация интерфейса с данными

При запуске приложения выполняется загрузка локально сохранённых данных и их отображение в таблице.

После инициализации интерфейса осуществляется асинхронная синхронизация с сервером посредством REST API. Полученные данные сохраняются в локальное хранилище и отображаются пользователю.

При добавлении, изменении или удалении задачи интерфейс обновляется автоматически, что обеспечивает актуальность отображаемой информации.

Для обновления содержимого используются механизмы перезагрузки источников данных компонентов `UITableView` и `UICollectionView`.

4.8. Особенности пользовательского интерфейса

Разработанный интерфейс обладает следующими особенностями:

- использование нативных компонентов iOS;
- поддержка навигации между экранами;
- отображение списка задач и категорий;
- использование адаптивной вёрстки на основе Auto Layout;
- поддержка операций создания, удаления и просмотра задач;
- автоматическое обновление интерфейса после синхронизации данных.

5. Организация работы клиента с локальным и серверным хранилищем данных

5.1. Общая схема хранения данных

Клиентское приложение реализует двухуровневую систему хранения данных, включающую локальное и удалённое хранилище.

Локальное хранение реализовано с использованием базы данных SQLite и предназначено для сохранения пользовательских задач непосредственно на мобильном устройстве. Это обеспечивает возможность доступа к данным без постоянного подключения к сети и ускоряет работу интерфейса.

Удалённое хранение реализовано посредством взаимодействия с серверным REST API, обеспечивающим централизованное хранение и синхронизацию задач.

Таким образом, клиентское приложение использует гибридную модель работы с данными, при которой локальная база обеспечивает быстрый доступ к информации, а серверная часть используется для централизованного хранения и обмена данными.

5.2. Работа с локальной базой данных

Для хранения задач на мобильном устройстве используется встроенная система управления базами данных SQLite.

Доступ к локальному хранилищу реализован классом `SQLiteTaskStore`, который отвечает за:

- создание базы данных;
- создание таблицы задач;

- получение списка задач;
- добавление и обновление записей;
- удаление задач.

Файл базы данных создаётся в каталоге документов приложения (`Documents`), что обеспечивает сохранение информации между запусками программы.

При инициализации объекта выполняется открытие соединения с базой данных и создание таблицы задач в случае её отсутствия.

5.3. Структура локальной базы данных

В локальной базе данных используется таблица `tasks`, предназначенная для хранения пользовательских задач.

Структура таблицы включает следующие поля:

Таблица 2: Структура таблицы `tasks`

Поле	Тип	Назначение
`id`	TEXT	Уникальный идентификатор задачи
`title`	TEXT	Название задачи
`notes`	TEXT	Текстовое описание
`is_done`	INTEGER	Статус выполнения задачи
`category`	TEXT	Категория задачи
`updated_at`	TEXT	Время последнего изменения

Поле `id` используется как первичный ключ таблицы, что гарантирует уникальность каждой записи.

Для хранения даты изменения используется строковое представление времени в формате ISO 8601. Такой подход обеспечивает совместимость при обмене данными между клиентом и сервером.

5.4. Операции над локальными данными

Работа с локальной базой данных осуществляется посредством SQL-запросов.

Получение списка задач выполняется с помощью запроса выборки:

- чтение записей из таблицы;
- сортировка по времени обновления;
- преобразование результатов в объекты приложения.

Добавление и обновление задач выполняется посредством операции `INSERT OR REPLACE`. Такой механизм позволяет использовать единый алгоритм как для создания новой записи, так и для изменения существующей.

Удаление задачи выполняется SQL-запросом `DELETE`, принимающим идентификатор задачи.

После выполнения SQL-запросов информация немедленно становится доступной пользовательскому интерфейсу, что позволяет обеспечить актуальное отображение данных.

5.5. Взаимодействие с серверным хранилищем

Обмен данными с сервером реализован отдельным компонентом `APIClient`, который отвечает за выполнение HTTP-запросов к REST API.

Для сетевого взаимодействия используется `URLSession`, входящий в состав платформы iOS.

Компонент поддерживает следующие операции:

- получение списка задач с сервера;
- добавление или обновление задачи;
- удаление задачи.

Все операции выполняются асинхронно, что предотвращает блокировку пользовательского интерфейса при ожидании ответа сервера.

5.6. Получение данных с сервера

Получение списка задач осуществляется посредством HTTP-запроса метода `GET` к маршруту:

```
/tasks
```

После получения ответа выполняется:

- проверка HTTP-кода состояния;
- декодирование JSON-документа;
- преобразование полученных данных в объекты приложения.

Для обработки даты используется формат ISO 8601, что обеспечивает единое представление времени как на сервере, так и на клиенте.

Полученные задачи сохраняются в локальную базу данных и становятся доступными для отображения в пользовательском интерфейсе.

5.7. Добавление и изменение задач на сервере

Для сохранения задачи на сервере используется HTTP-метод `POST`.

Клиент сериализует объект задачи в JSON-документ и передаёт его серверу вместе с HTTP-запросом.

Запрос включает заголовок:

```
Content-Type: application/json
```

После успешного выполнения операции сервер возвращает код состояния `201 Created`, подтверждающий сохранение объекта.

При изменении задачи используется тот же механизм, поскольку серверная часть реализует объединённую операцию добавления и обновления.

5.8. Удаление данных с сервера

Удаление задачи выполняется посредством HTTP-запроса метода `DELETE`.

В адресе запроса передаётся уникальный идентификатор задачи.

После успешного удаления сервер возвращает код состояния:

```
204 No Content
```

Если задача отсутствует на сервере, используется код:

```
404 Not Found
```

Подобный подход позволяет клиентскому приложению корректно обрабатывать ситуации рассинхронизации данных.

5.9. Механизм синхронизации данных

При запуске приложения сначала загружаются данные из локальной базы SQLite, после чего выполняется асинхронный запрос к серверу.

Данный подход обеспечивает быстрое отображение информации пользователю даже при нестабильном интернет-соединении.

Алгоритм синхронизации можно представить следующим образом:

1. загрузка данных из SQLite;
2. отображение задач в интерфейсе;
3. выполнение запроса к REST API;
4. получение актуальных данных с сервера;
5. сохранение данных в локальную базу;
6. обновление пользовательского интерфейса.

При добавлении, редактировании или удалении задачи изменения сначала сохраняются локально, после чего асинхронно отправляются на сервер.

Такой подход позволяет минимизировать задержки пользовательского интерфейса и обеспечивает возможность частичной работы приложения при отсутствии сети.

6. Список источников

1. Apple Developer Documentation: UIKit [Электронный ресурс] / Режим доступа: https://developer.apple.com/documentation/uikit/, свободный — 18.05.2026.

2. Apple Developer Documentation: URLSession [Электронный ресурс] / Режим доступа: https://developer.apple.com/documentation/foundation/urlsession свободный — 18.05.2026.

3. Apple Developer Documentation: SQLite3 API [Электронный ресурс] / Режим доступа: https://developer.apple.com/library/archive/documentation/Gene свободный — 18.05.2026.

4. Vapor Framework Documentation [Электронный ресурс] / Режим доступа: https://docs.vapor.codes/, свободный — 18.05.2026.

5. SQLite Documentation [Электронный ресурс] / Режим доступа: https://www.sqlite.org/docs.html, свободный — 18.05.2026.

7. Вывод

В ходе выполнения курсового проекта было разработано мобильное приложение “Todo” для платформы iOS, предназначенное для создания, хранения и управления пользовательскими задачами.

В процессе разработки была спроектирована и реализована серверная часть приложения, предоставляющая REST API для централизованного хранения данных. Сервер обеспечивает выполнение основных операций над задачами, включая создание, получение, обновление и удаление записей, а также поддерживает обмен данными в формате JSON.

Была реализована клиентская часть приложения с использованием средств платформы iOS и фреймворка UIKit. Разработанный графический интерфейс включает экран списка задач, экран категорий и дополнительные средства взаимодействия с пользовательскими данными. При проектировании интерфейса использовались стандартные компоненты платформы, включая `UITableViewController` и `UICollectionViewController`, а построение интерфейса выполнено с применением Auto Layout и системы ограничений.

Для хранения информации на мобильном устройстве реализована локальная база данных SQLite, обеспечивающая сохранность данных между запусками приложения и возможность быстрого доступа к ним. Дополнительно реализовано взаимодействие с удалённым серверным хранилищем посредством REST API, что позволило организовать синхронизацию данных между клиентом и сервером.

В рамках проекта были реализованы механизмы асинхронной обработки сетевых запросов, сериализации данных и обновления пользовательского интерфейса после изменения состояния хранилища.

В результате выполнения работы были выполнены основные тре-

бования, предъявляемые к курсовому проекту: реализовано мобильное приложение для iOS, разработан собственный сервер с REST API, использована локальная база данных SQLite, применены `TableViewController` и `CollectionViewController`, реализована адаптивная вёрстка с использованием Auto Layout и обеспечено централизованное хранение пользовательских данных.

Поставленная цель разработки мобильного приложения для управления задачами была достигнута.